

Quai Antique

Site web du restaurant gastronomique

Larisa Faessel

Développeur No Code (DNC)

Studi · Cohorte Septembre – Octobre 2026

Dossier Projet · Juin 2026



Sommaire

01

Contexte & objectifs

Slides 3–4

02

Cahier des charges

Slides 5–10

03

Maquettage & Conception

Slides 11–13

04

Environnement & Architecture

Slides 14–20

05

Base de données

Slides 21–25

06

Backend — API & Auth

Slides 26–32

07

Frontend — Interface

Slides 33–42

08

Déploiement

Slides 43–46

09

Démonstration

Slide 47

10

Validation & Tests

Slides 48–49

11

Bilan & Évolution

Slides 50–56

12

Compétences DNC

Slides 57–60

Le restaurant Quai Antique

Contexte du projet

- ▶ Restaurant gastronomique fictif situé à Chambéry
- ▶ Chef : Arnaud Michant
- ▶ Cuisine de saison, produits locaux et frais
- ▶ Service midi et soir, du mardi au dimanche
- ▶ Absence de présence numérique moderne
- ▶ Besoin : permettre aux clients de réserver en ligne

Projet réalisé dans le cadre du Dossier Projet DNC · Studi · 2026



Objectifs du projet

01 Vitrine en ligne

Présenter le restaurant, la galerie photo et la carte gastronomique complète

02 Réservation en ligne

Formulaire connecté à une API REST et une base de données réelle

03 Espace utilisateur

Inscription, connexion sécurisée, préremplissage automatique du formulaire

04 Espace administrateur

Dashboard de consultation des réservations avec indicateurs de statut

02

Cahier des charges

Besoins fonctionnels — Visiteur

Fonctionnalité	Description	✓
Accueil restaurant	Présentation du chef, des valeurs, image hero	✓
Galerie photos	6 images du restaurant et des plats	✓
Carte du restaurant	Entrées, plats, desserts avec prix et descriptions	✓
Navigation par onglets	Passage entre catégories sans rechargement de page	✓
Formulaire de réservation	Envoi date, heure, convives, allergies	✓
Horaires & contact	Footer visible sur toutes les pages	✓
Responsive mobile	Adaptation complète de 390px à 1280px	✓

Besoins fonctionnels — Utilisateur connecté

Fonctionnalité	✓
S'inscrire (prénom, nom, email, mot de passe)	✓
Se connecter avec ses identifiants	✓
Formulaire de réservation prérempli automatiquement	✓
Navigation affiche le prénom de l'utilisateur	✓
Se déconnecter en un clic	✓
Redirection vers la réservation après connexion	✓

Quai Antique

AccueilGalerieLa carteLes reservationsMon compte

Réserver votre repas

Prénom

Marie

Nom

Dupont

Email

marie.dupont@example.com

Date

dd/mm/yyyy

Heure

--:--

Nombre de personnes

1 personne

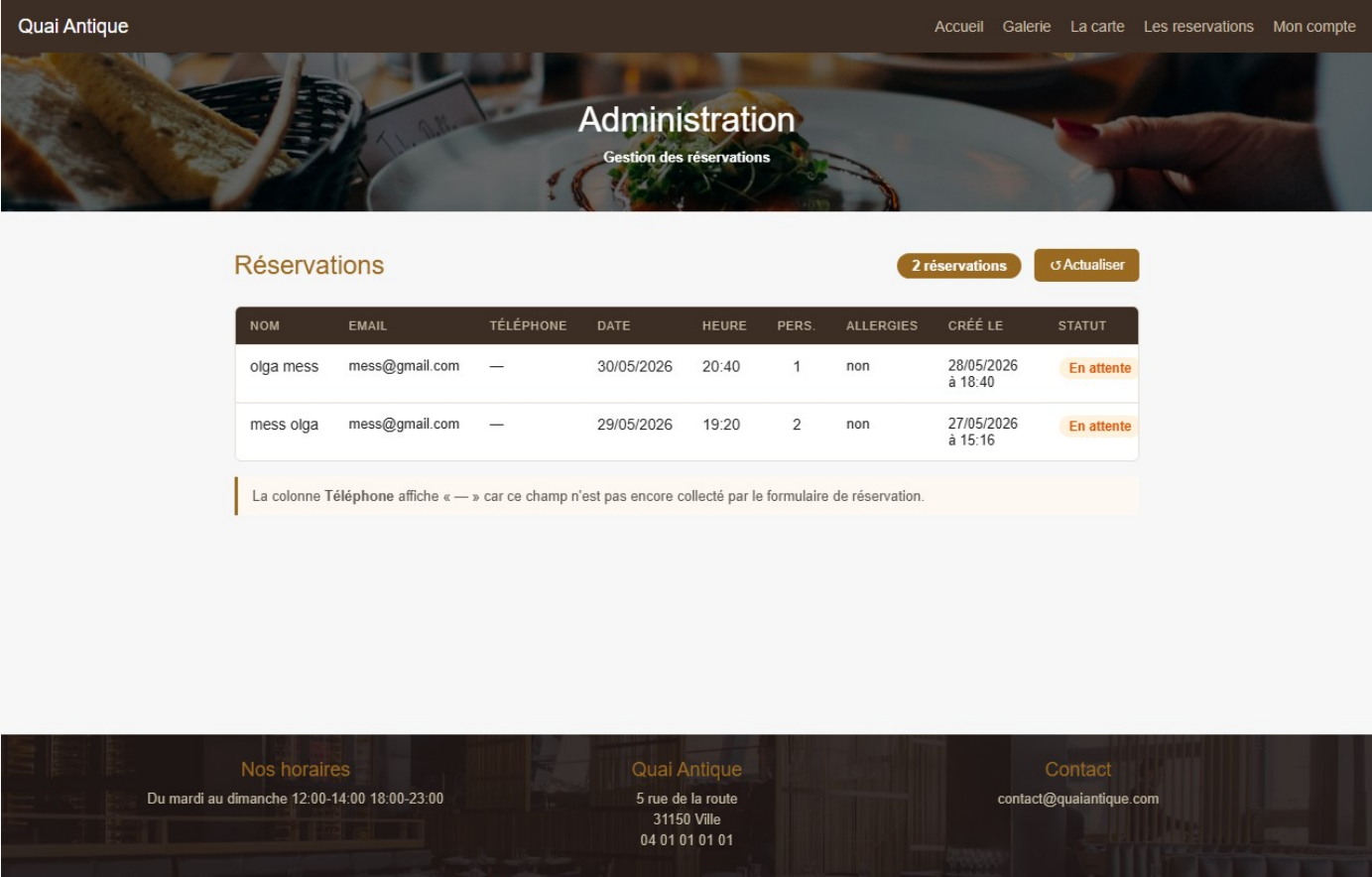
Allergies

Message

Formulaire prérempli automatiquement après connexion

7 / 60

Besoins fonctionnels — Administrateur



Dashboard Administrateur

- Consultation de toutes les réservations
- Tri automatique par date (plus récente en premier)
- Indicateurs de statut colorés (En attente / Confirmée / Annulée)
- Colonnes : nom, email, date, heure, convives, allergies
- Bouton Actualiser pour rechargement en temps réel
- Accès via la page /admin.html

✓ Consultation centralisée des réservations opérationnelle

Besoins non fonctionnels

Critère	Exigence	Solution	✓
Responsive	Mobile 390px + Desktop 1280px	CSS Flexbox / Grid + media query	✓
Sécurité	Hashage irréversible des mots de passe	bcrypt 10 rounds	✓
Authentification	Sessions sécurisées et stateless	JWT · 7 jours	✓
Protection données	Variables secrètes hors code source	dotenv · .gitignore	✓
CORS	API accessible uniquement par le frontend	Variable d'environnement CORS_ORIGIN	✓
Disponibilité	Site accessible 24h/24, 7j/7	Netlify CDN + Railway	✓
Versioning	Historique et traçabilité du code	Git + GitHub · 20 commits	✓
Déploiement automatique	Mise en ligne à chaque modification	GitHub → Netlify / Railway	✓

Mapping besoins → implémentation

Besoin client	Implémentation	Fichier(s)	✓
Présenter le restaurant	Pages HTML statiques + CSS	index.html · styles.css	✓
Afficher la carte	Onglets dynamiques JavaScript	menu.html · app.js	✓
Réserver une table	Formulaire → API → PostgreSQL	reservation.html · controller	✓
Créer un compte	Inscription avec sécurisation bcrypt	auth.controller · user.repo	✓
Se connecter	Authentification JWT	auth.controller · app.js	✓
Formulaire prérempli	Lecture localStorage → champs formulaire	app.js	✓
Gérer les réservations	Dashboard admin avec indicateurs	admin.html	✓
Site sur mobile	Design responsive CSS	styles.css	✓
Mise en ligne	CI/CD automatique GitHub	Netlify + Railway	✓

03

Maquettage & Conception

Wireframes — Page d'accueil & Formulaire de réservation

Accueil

LOGO

Navigation

[Image Hero — Titre restaurant — Bouton Réserver]

Bienvenue — Texte + Image

Produits frais — Image + Texte

Réservation en ligne — Bouton

Horaires · Adresse · Contact

Réservation

LOGO

Navigation

Réserver votre repas

Prénom

Nom

Email

Date · Heure

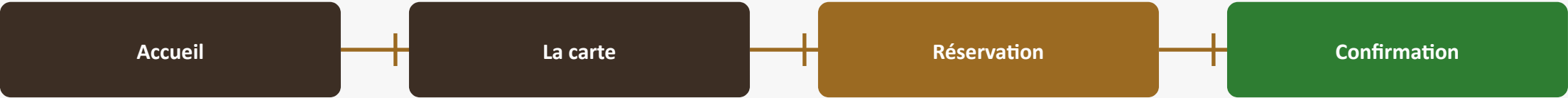
Nombre de personnes

Allergies (optionnel)

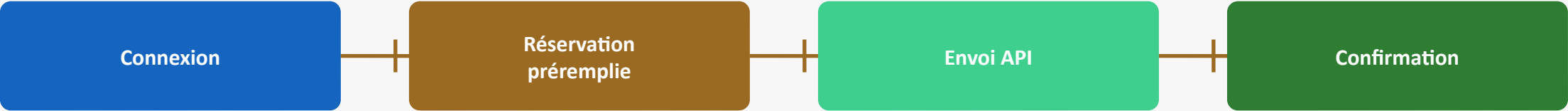
Réserver

Parcours utilisateur — Conception

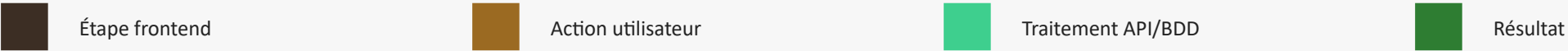
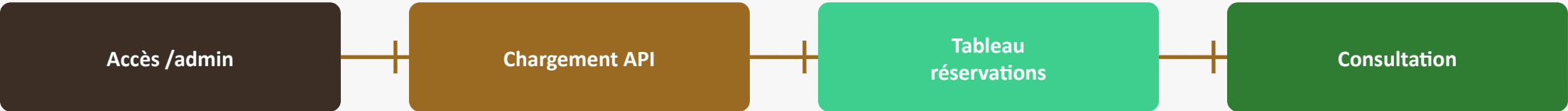
Parcours réservation (utilisateur non connecté)



Parcours réservation (utilisateur connecté)



Parcours administrateur



04

Environnement & Architecture

Environnement de travail

VS Code

Éditeur de code

Git

Versioning local

GitHub

Dépôt distant & CI/CD

Node.js

Runtime JavaScript backend

npm

Gestionnaire de dépendances

Postman

Test des routes API

Chrome

Navigateur & DevTools

Supabase

Base de données PostgreSQL

Stack technique

Frontend

- HTML5
- CSS3
- JavaScript ES6+
- Vanilla (sans framework)
- styles.css · 400 lignes
- app.js · 178 lignes

Backend

- Node.js 24
- Express 4
- bcryptjs
- jsonwebtoken (JWT)
- dotenv
- pg (node-postgres)

Base de données

- PostgreSQL
- Supabase (cloud)
- Requêtes préparées
- 2 tables (users + reservations)
- Initialisation automatique
- SSL activé

Déploiement

- Netlify (frontend)
- Railway (backend)
- GitHub CI/CD
- Nixpacks builder
- Docker + Nginx (local)
- HTTPS automatique

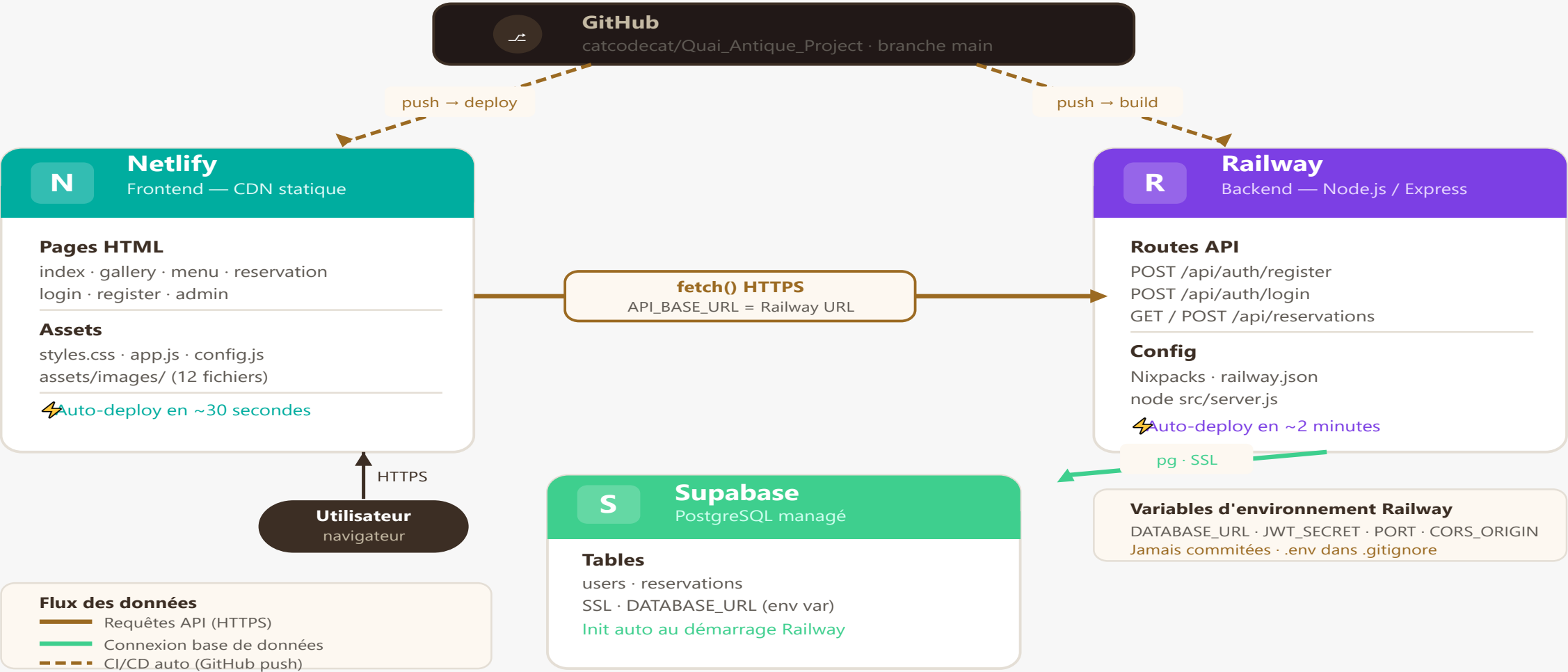
Justification des choix techniques

Technologie	Pourquoi ce choix ?
HTML / CSS / JS vanilla	Maîtrise complète du code · Idéal pour apprendre les fondamentaux web
Node.js + Express	JavaScript côté serveur · Léger · Parfait pour une API REST
PostgreSQL / Supabase	Base relationnelle robuste · Gratuit · Gestion cloud simplifiée
JWT	Authentification légère, sécurisée et stateless (sans session serveur)
bcrypt	Standard de référence pour le hashage sécurisé des mots de passe
Netlify	Déploiement frontend automatique en 30 secondes · CDN mondial · HTTPS
Railway	Déploiement backend Node.js simple · Variables d'env sécurisées
Git / GitHub	Versioning essentiel · CI/CD automatique · Traçabilité complète

Architecture globale de l'application

Architecture Globale — Quai Antique

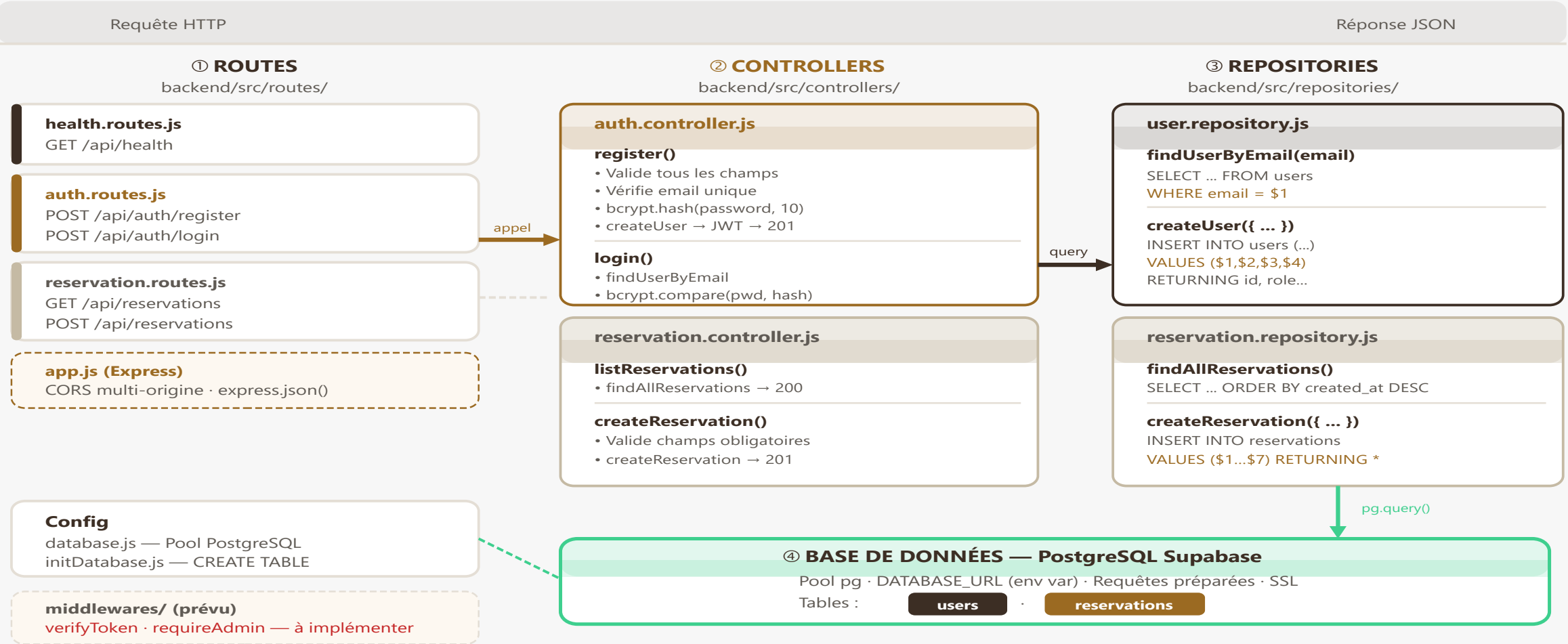
Netlify · Railway · Supabase PostgreSQL



Pattern MVC — Organisation du backend

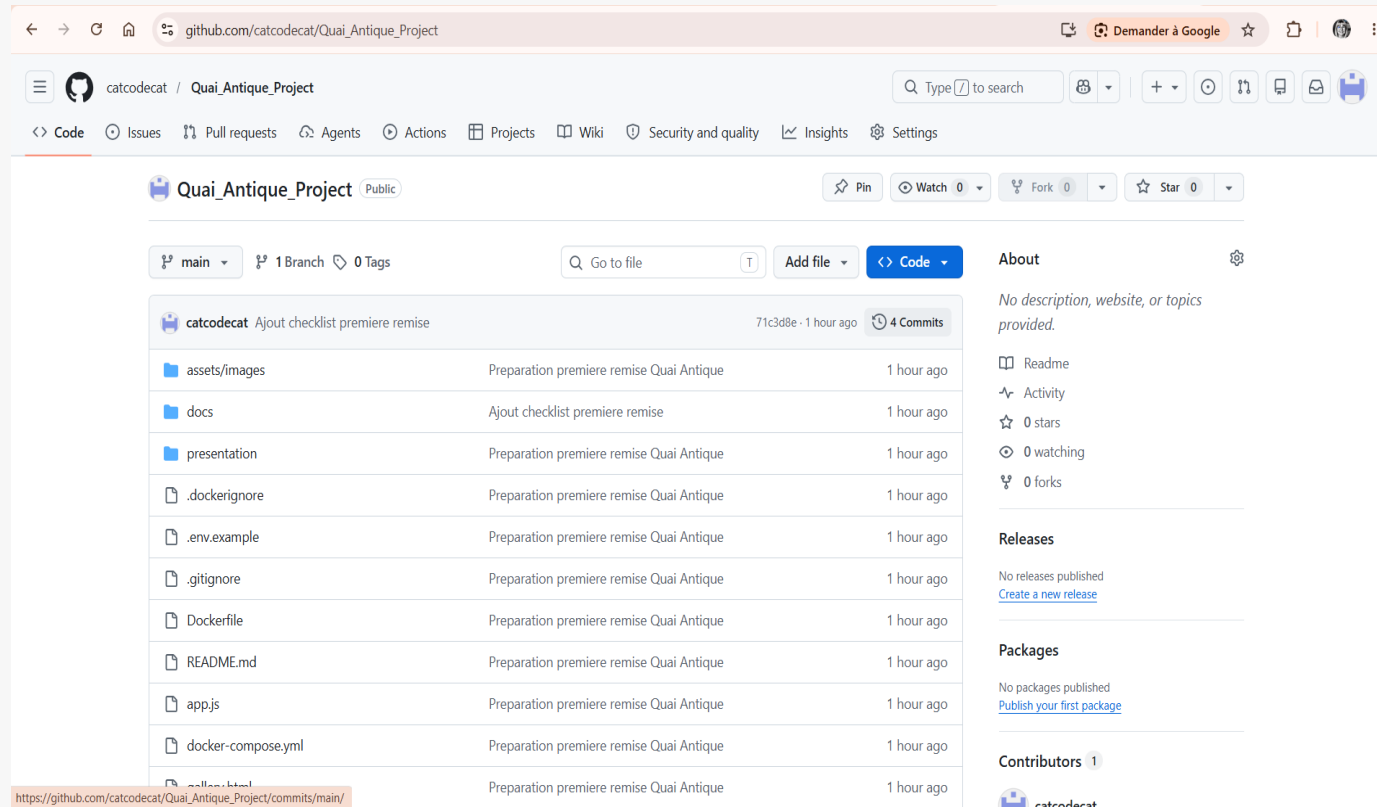
Pattern MVC — Backend Quai Antique

Routes · Controllers · Repositories · Base de données



Règle MVC : les Routes ne contiennent pas de logique · les Controllers ne font pas de SQL · les Repositories ne font que du SQL

Git & GitHub — Versioning & CI/CD



20 commits documentés

- Messages de commit clairs et structurés
- Conventions : feat: / fix: / add:
- Push → déploiement Netlify automatique (~30s)
- Push → rebuild Railway automatique (~2min)
- Fichiers sensibles exclus du dépôt (.env)
- Historique complet et traçable

05

Base de données

Modèle Entité-Relation (ERD)

Base de données — Modèle Entité-Relation

Projet Quai Antique · PostgreSQL / Supabase

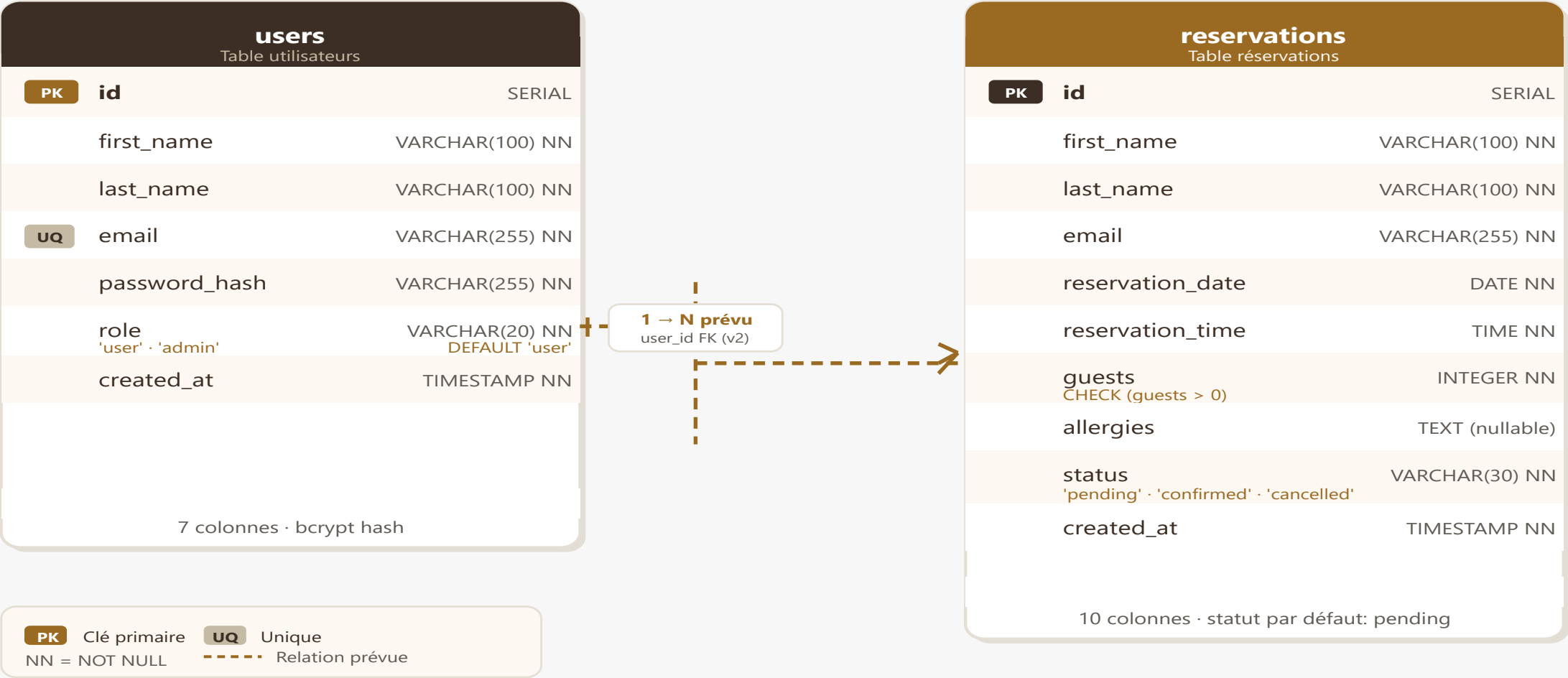


Table users — Structure & sécurité

```
CREATE TABLE IF NOT EXISTS users (  
  id          SERIAL PRIMARY KEY,  
  first_name  VARCHAR(100) NOT NULL,  
  last_name   VARCHAR(100) NOT NULL,  
  email       VARCHAR(255) UNIQUE NOT NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  role        VARCHAR(20)  DEFAULT 'user',  
  created_at  TIMESTAMP    DEFAULT NOW()  
);
```

Points clés

SERIAL PK

Identifiant auto-incrémenté

UNIQUE email

Un seul compte par adresse

password_hash

Mot de passe jamais stocké en clair

role DEFAULT 'user'

Deux rôles : user et admin

Créé automatiquement

Tables générées au démarrage

Table reservations — Statuts & contraintes

```
CREATE TABLE IF NOT EXISTS reservations (  
  id          SERIAL PRIMARY KEY,  
  first_name  VARCHAR(100) NOT NULL,  
  last_name   VARCHAR(100) NOT NULL,  
  email       VARCHAR(255) NOT NULL,  
  reservation_date DATE      NOT NULL,  
  reservation_time TIME      NOT NULL,  
  guests      INTEGER CHECK (guests > 0),  
  allergies   TEXT,  
  status      VARCHAR(30)  DEFAULT 'pending',  
  created_at  TIMESTAMP    DEFAULT NOW()  
);
```

Statuts de réservation

pending → En attente

confirmed → Confirmée

cancelled → Annulée

Requêtes préparées — Sécurité SQL

```
// user.repository.js - Protection injection SQL

// ✅ Requête préparée - paramètre $1 séparé
const result = await db.query(
  `SELECT id, first_name AS "firstName", email,
    password_hash AS "passwordHash"
  FROM users
  WHERE email = $1`,
  [email] // ← valeur traitée comme donnée
);

// reservation.repository.js
await db.query(
  `INSERT INTO reservations (first_name, email, ...)
  VALUES ($1, $2, $3, $4, $5, $6, $7)
  RETURNING *`,
  [firstName, email, date, time, guests, allergies]
);
```

Principe de sécurité

- ▶ Les données utilisateur ne sont jamais concaténées dans le SQL
- ▶ Le paramètre \$1, \$2... est traité comme une donnée, pas comme du code
- ▶ Protection contre les injections SQL (OWASP Top 10)
- ▶ Appliqué dans tous les repositories du projet

06

Backend API & Authentication

Routes API — Quai Antique

Base URL : `https://quaiantiqueproject-production.up.railway.app`

Méthode	Route	Description
GET	/api/health	Vérification de la disponibilité du serveur
POST	/api/auth/register	Créer un compte utilisateur (bcrypt + JWT)
POST	/api/auth/login	Connexion et obtention d'un token JWT
GET	/api/reservations	Consulter les réservations (dashboard admin)
POST	/api/reservations	Créer une nouvelle réservation

✓ API REST opérationnelle · Gestion des réservations et de l'authentification

Flux d'authentification — register · login · session



✓ Session active · Token JWT (7 jours) · Formulaire de réservation prérempli automatiquement

JSON Web Token (JWT) — Authentification sécurisée

```
// auth.controller.js
const token = jwt.sign(
  { userId: user.id, email: user.email, role: user.role },
  process.env.JWT_SECRET,
  { expiresIn: '7d' }
);

// app.js (frontend)
localStorage.setItem("qaToken", token);
localStorage.setItem("qaUser", JSON.stringify(user));
```

Ce que cela apporte

► Session sans stockage serveur (stateless)

► Token contient userId, email, rôle

► Expiration automatique après 7 jours

► Sécurisé par une clé secrète (JWT_SECRET)

Gestion des réservations — Parcours complet

```
// reservation.controller.js
async function createReservation(req, res) {
  const { firstName, lastName, email,
        date, time, guests, allergies } = req.body;

  // Validation des champs obligatoires
  if (!firstName || !email || !date || !time || !guests)
    return res.status(400).json({ message: 'Champs manquants' });

  // Insertion en base
  const reservation = await reservationRepository
    .createReservation({ firstName, lastName, email,
                        date, time, guests, allergies });

  return res.status(201).json({ reservation });
}
```

Parcours de réservation

- 1 Formulaire rempli par l'utilisateur
- 2 fetch() POST → API Railway
- 3 Validation des données côté serveur
- 4 INSERT SQL préparé → PostgreSQL
- 5 Réservation créée (statut : En attente)
- 6 Réponse 201 → message de succès
- 7 Visible dans le dashboard admin

Sécurité — Mesures implémentées

Mesure	Implémentation	✓
Hash mots de passe	bcrypt 10 rounds · irréversible	✓
Authentification	JWT signé · expiration 7 jours	✓
Variables secrètes	dotenv · .env exclu du dépôt Git	✓
Protection SQL	Requêtes préparées (\$1, \$2...)	✓
CORS	Origines autorisées via variable d'environnement	✓
HTTPS	Automatique sur Netlify et Railway	✓
Validation des entrées	Vérification côté serveur dans les controllers	✓
Tests fonctionnels	Réalisés et validés sur le site de production	✓

✓ Toutes les mesures de sécurité essentielles sont implémentées et opérationnelles

CORS & Variables d'environnement

CORS — Contrôle des origines autorisées

```
// backend/src/app.js
const allowedOrigins =
  (process.env.CORS_ORIGIN || 'http://localhost:8080')
    .split(',')
    .map(o => o.trim());

app.use(cors({
  origin: (origin, callback) => {
    if (!origin || allowedOrigins.includes(origin))
      callback(null, true); // ✅ autorisé
    else
      callback(new Error('Non autorisé'));
  }
}));
```

Variables d'environnement Railway

DATABASE_URL

Connexion PostgreSQL Supabase

JWT_SECRET

Signature des tokens JWT

PORT

Port Express (3000)

CORS_ORIGIN

Domaines autorisés à appeler l'API

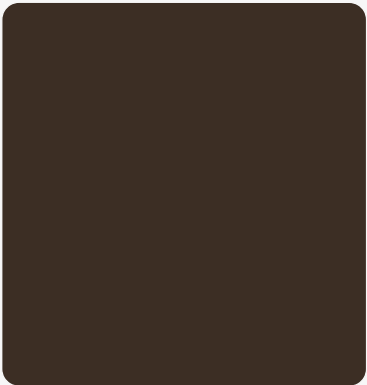
✓ Aucune donnée sensible dans le code source

07

Frontend Interface utilisateur

Charte graphique

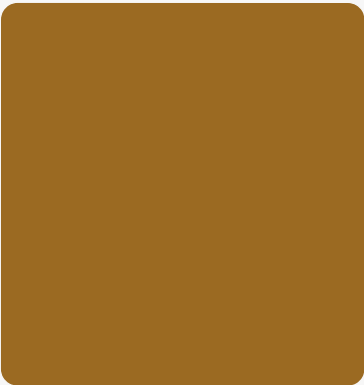
Palette utilisée dans l'application Quai Antique



#3C2E24

Brun foncé

Header · titres



#9B6A22

Or · Principal

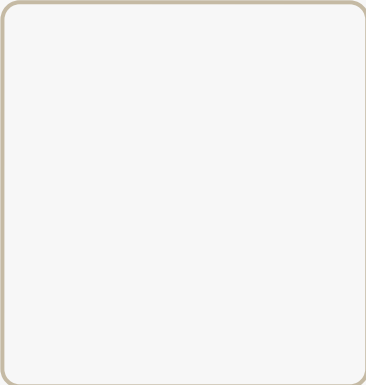
Boutons · accents



#C5BAA4

Beige

Textes secondaires



#F7F7F7

Gris clair

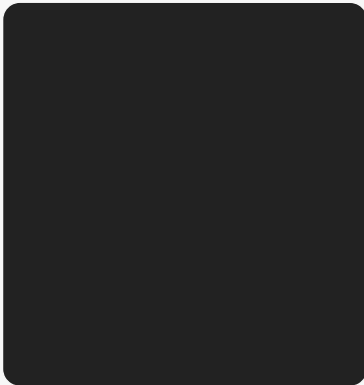
Fond des pages



#211817

Noir brun

Sections sombres



#222222

Anthracite

Texte courant

Typographies

Titres : Montserrat — font-weight: 500

Corps : Hind Madurai

Bouton principal

Réserver

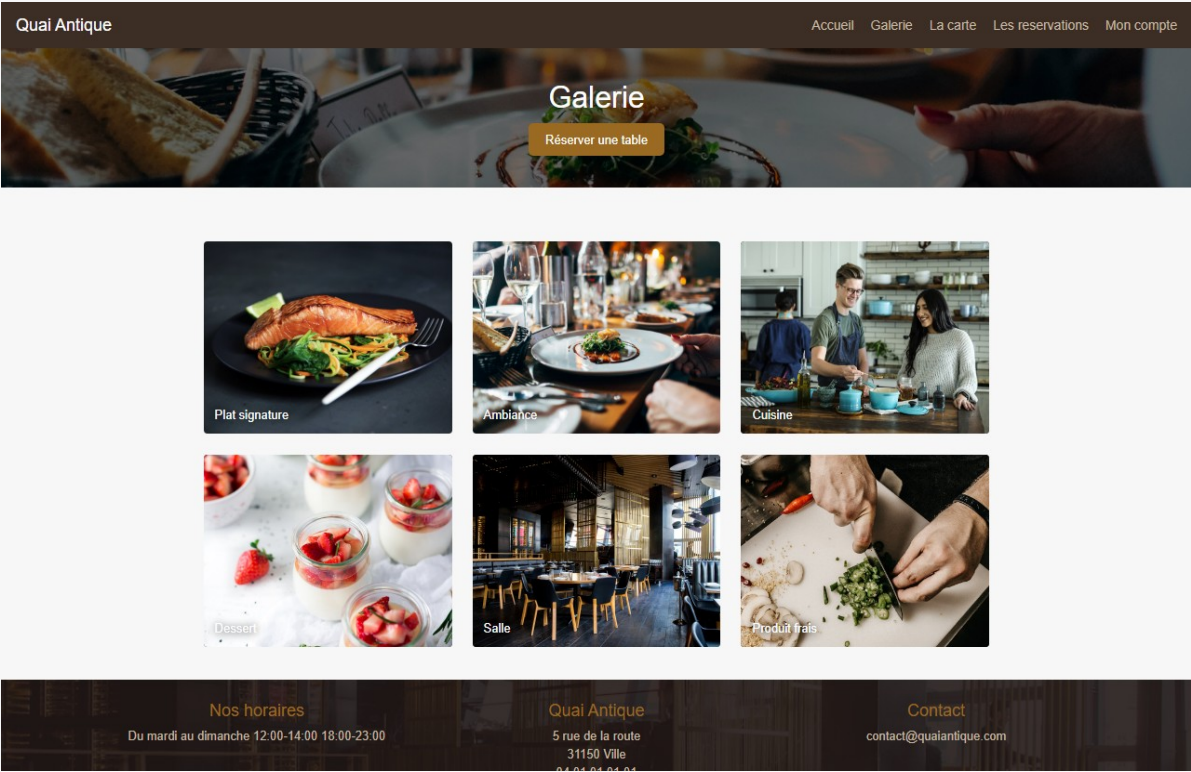
Objectif : ambiance gastronomique haut de gamme, chaleureuse et élégante.

Pages statiques — Accueil & Galerie



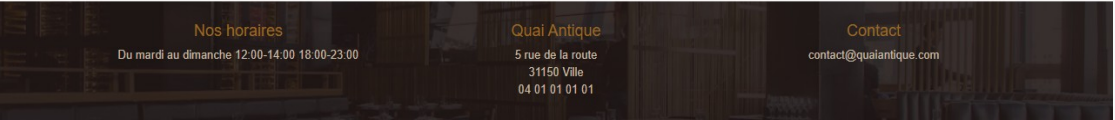
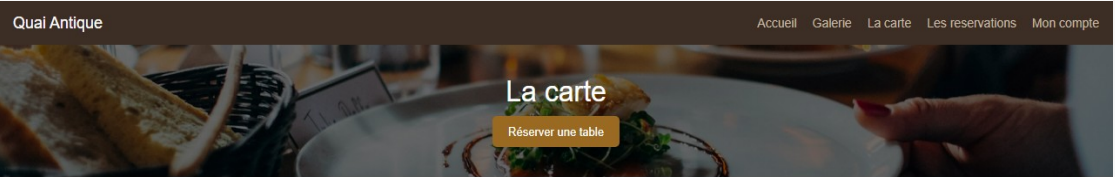
index.html

- Header + nav identiques sur toutes les pages
- Hero avec image plein largeur
- Sections alternées clair/foncé
- Boutons CTA vers réservation
- Footer : horaires, adresse, contact

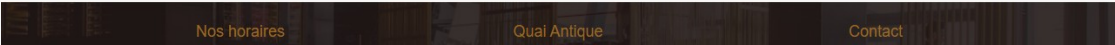
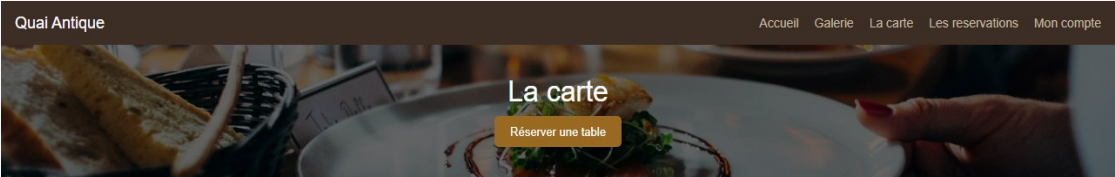


gallery.html

La carte — Navigation par onglets



Onglet Entrées

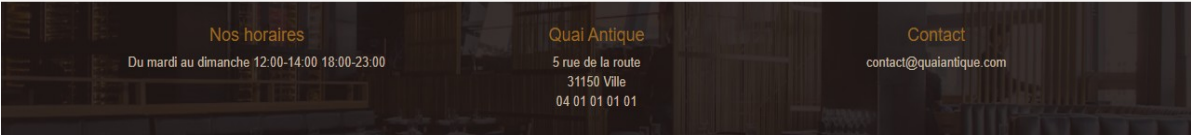
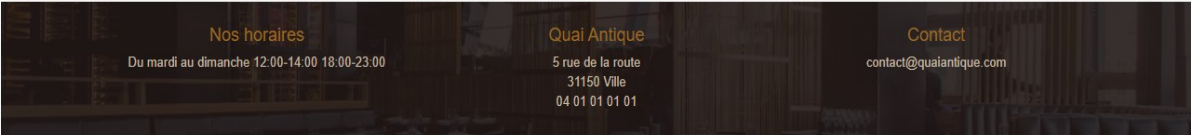
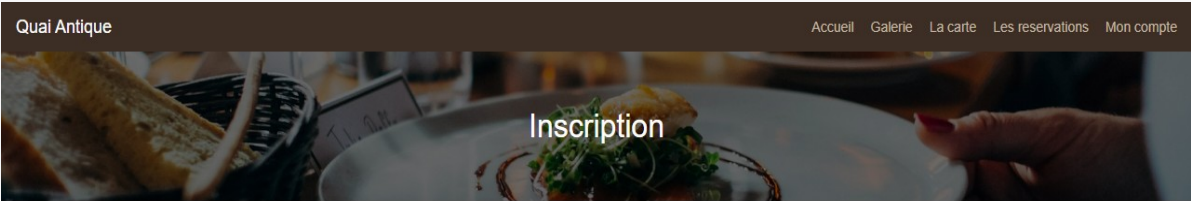
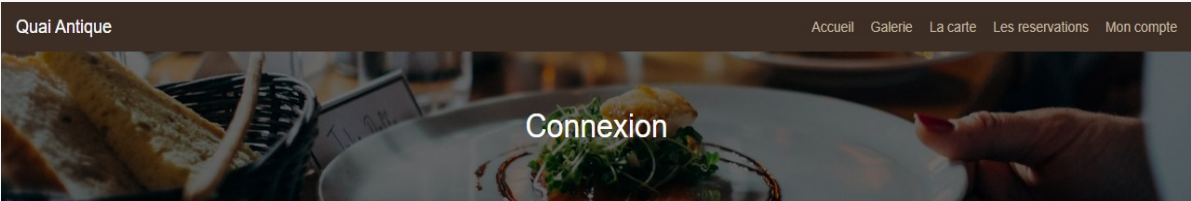


Onglet Plats

Implémentation JavaScript : fonction showMenuPanel()

```
classList.toggle('active') · classList.toggle('is-visible') · history.replaceState() · data-tab / data-panel
```

Formulaires — Connexion & Inscription



login.html

register.html

Messages de validation affichés · Redirection automatique vers la réservation après connexion

Session utilisateur — Gestion de la connexion



La navigation affiche «Prénom — Déconnexion» quand l'utilisateur est connecté

```
function updateNav() {
  const user = JSON.parse(
    localStorage.getItem("qaUser") || "null");

  if (user) {
    navLink.textContent =
      `${user.firstName} — Déconnexion`;
    navLink.addEventListener("click", () => {
      localStorage.removeItem("qaToken");
      localStorage.removeItem("qaUser");
      window.location.href = "index.html";
    });
  }
}
```

Préremplissage automatique du formulaire



Prénom

Nom

Email

Date

Heure

Nombre de personnes

Allergies

Prénom, nom et email préremplis automatiquement dès la connexion

```
// app.js - réservation.html
const user = JSON.parse(
  localStorage.getItem("qaUser") || "null"
);

if (user) {
  document.getElementById("firstName").value
    = user.firstName || "";
  document.getElementById("lastName").value
    = user.lastName || "";
  document.getElementById("email").value
    = user.email || "";
}
```


Dashboard Administrateur



Réservations

2 réservations

Actualiser

NOM	EMAIL	TÉLÉPHONE	DATE	HEURE	PERS.	ALLERGIES	CRÉÉ LE	STATUT
olga mess	mess@gmail.com	—	30/05/2026	20:40	1	non	28/05/2026 à 18:40	En attente
mess olga	mess@gmail.com	—	29/05/2026	19:20	2	non	27/05/2026 à 15:16	En attente

La colonne Téléphone affiche « — » car ce champ n'est pas encore collecté par le formulaire de réservation.



✓ Tableau d'administration opérationnel • Consultation centralisée des réservations

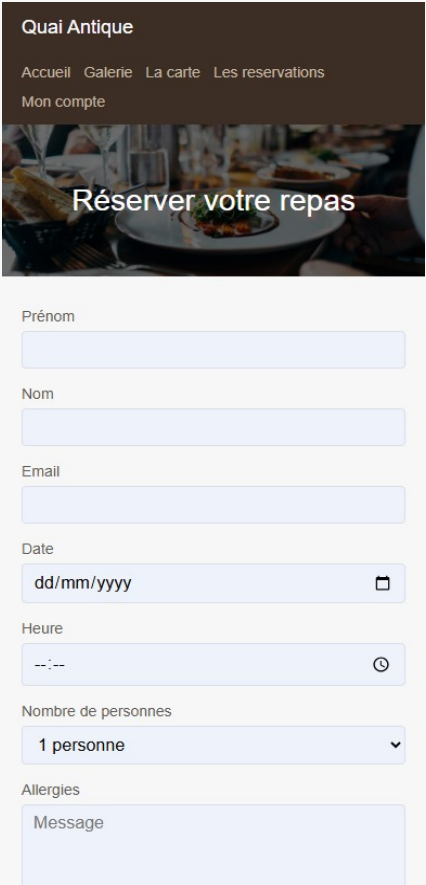
Responsive Design — Mobile 390×844



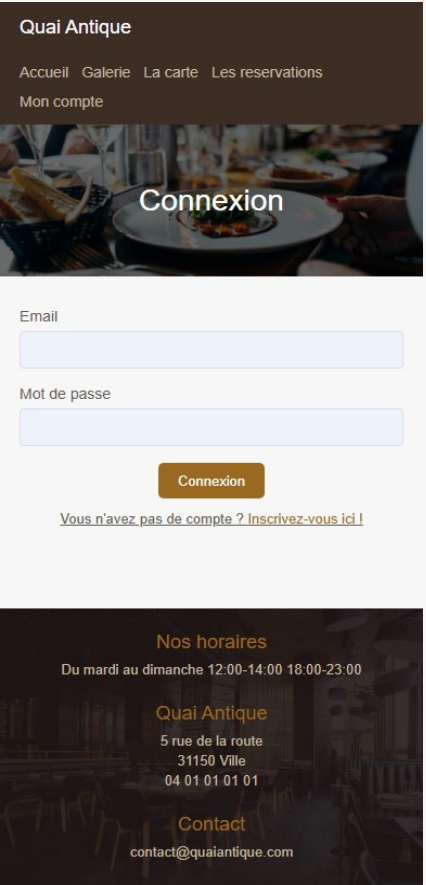
Accueil



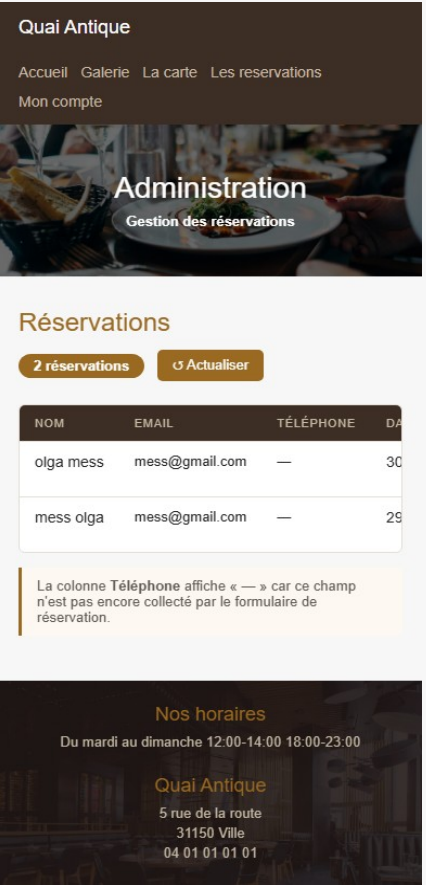
Carte



Réservation



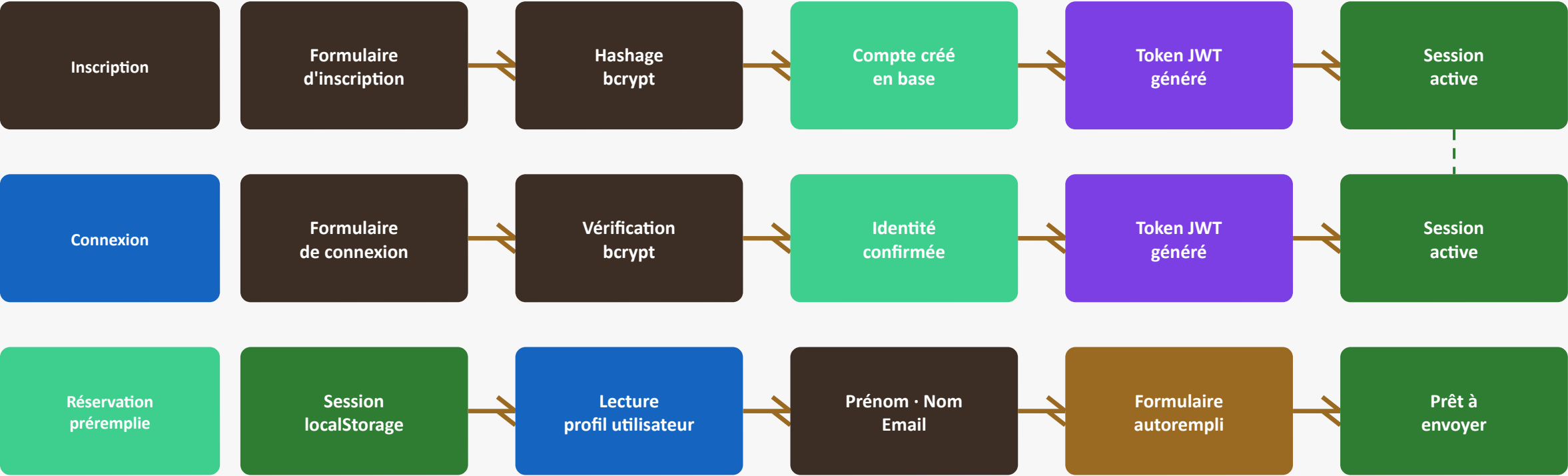
Connexion



Admin

CSS Flexbox + Grid + media query · Rendu à 390px (iPhone 14)

Parcours utilisateur — Scénario nominal



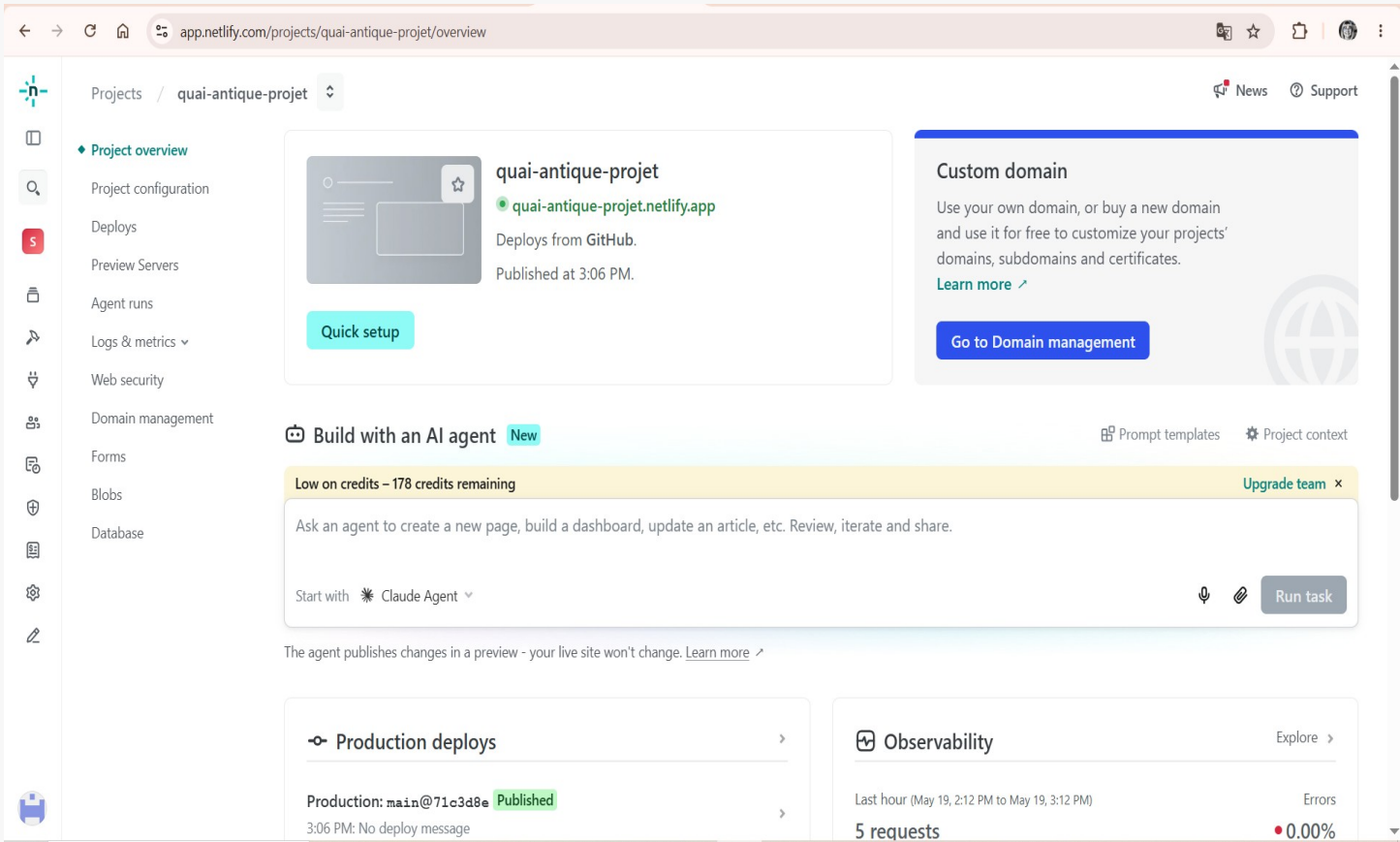
✓ Résultat — Parcours utilisateur validé

- ✓ Token JWT stocké en localStorage
- ✓ Navigation affiche le prénom
- ✓ Formulaire prérempli automatiquement
- ✓ Réservation envoyée avec succès

08

Déploiement

Netlify — Déploiement Frontend



Points clés

- ▶ quai-antique-projet.netlify.app
- ▶ Déploiement automatique à chaque push
- ▶ HTTPS activé automatiquement
- ▶ CDN mondial · disponible partout
- ▶ Déploiement en ~30 secondes
- ▶ Plan gratuit

Railway — Déploiement Backend Node.js

railway.json

```
{
  "build": { "builder": "NIXPACKS" },
  "deploy": {
    "startCommand": "node src/server.js",
    "restartPolicyType": "ON_FAILURE"
  }
}
```

Problèmes résolus pendant le déploiement

- ✓ Variable PORT → process.env.PORT intégré
- ✓ Builder → Switch vers Nixpacks
- ✓ Health endpoint → Route /api/health ajoutée

Points clés

- ▶ API disponible en production
- ▶ Déploiement auto depuis GitHub (~2min)
- ▶ Variables d'env sécurisées dans Railway
- ▶ Restart automatique en cas d'erreur

Supabase (BDD cloud) & Docker (local)

Supabase — PostgreSQL cloud

- ▶ PostgreSQL managé, sans serveur à gérer
- ▶ Connexion via DATABASE_URL (variable sécurisée)
- ▶ SSL activé · Backups automatiques
- ▶ Tables créées automatiquement au démarrage
- ▶ Interface visuelle pour inspecter les données

Docker — Lancement local

```
# Frontend via Nginx
docker compose up --build
# → http://localhost:8080

# Backend en développement
cd backend && npm run dev
```

Utilité du Docker

- ▶ Tester le site sans dépendances locales
- ▶ Environnement identique pour tous
- ▶ Isolation du frontend (Nginx)

Démonstration du projet

Site Netlify

`quai-antique-projet.netlify.app`

Accueil · Galerie · Carte · Réservation
Login · Inscription · Admin

API Railway

`.../api/health → { status: 'ok' }`

GET /api/health
GET & POST /api/reservations
POST /api/auth/register & login

Dépôt GitHub

`github.com/catcodecat/Quai_Antique_Project`

20 commits · branche main
Historique complet du développement

Compte de démonstration

`demo@quaiantique.com · demo123`

Créer avant la soutenance
Parcours : login → réservation préremplie

Parcours à montrer : Accueil → Register → Login → Réservation préremplie → Dashboard Admin

Validation fonctionnelle

Fonctionnalité	Résultat	Statut
Inscription utilisateur	Compte créé · token JWT retourné · redirection	✓ Validée
Connexion utilisateur	Authentification réussie · session active	✓ Validée
Réservation en ligne	Données enregistrées en base PostgreSQL	✓ Validée
Préremplissage formulaire	Champs remplis automatiquement depuis la session	✓ Validé
API REST	Toutes les routes répondent correctement	✓ Validée
Dashboard administrateur	Réservations affichées avec indicateurs	✓ Validé
Responsive mobile	Affichage correct sur iPhone 14 (390px)	✓ Validé
Déploiement Netlify	Site accessible en production · HTTPS	✓ Validé
Déploiement Railway	API accessible en production · auto-deploy	✓ Validé
Base PostgreSQL	Tables créées · données persistées · requêtes OK	✓ Validée

✓ 10 fonctionnalités clés validées — projet opérationnel en production

Tests réalisés

Tests fonctionnels manuels réalisés sur le site de production

Authentification

- ✓ Création d'un compte avec données valides
- ✓ Tentative d'inscription avec email déjà utilisé → erreur affichée
- ✓ Connexion avec identifiants corrects
- ✓ Connexion avec mauvais mot de passe → erreur affichée
- ✓ Déconnexion et vérification de la session supprimée

Réservation

- ✓ Soumission d'une réservation complète → enregistrée en base
- ✓ Préremplissage automatique après connexion
- ✓ Envoi sans champs obligatoires → message d'erreur
- ✓ Réservation visible immédiatement dans le dashboard admin

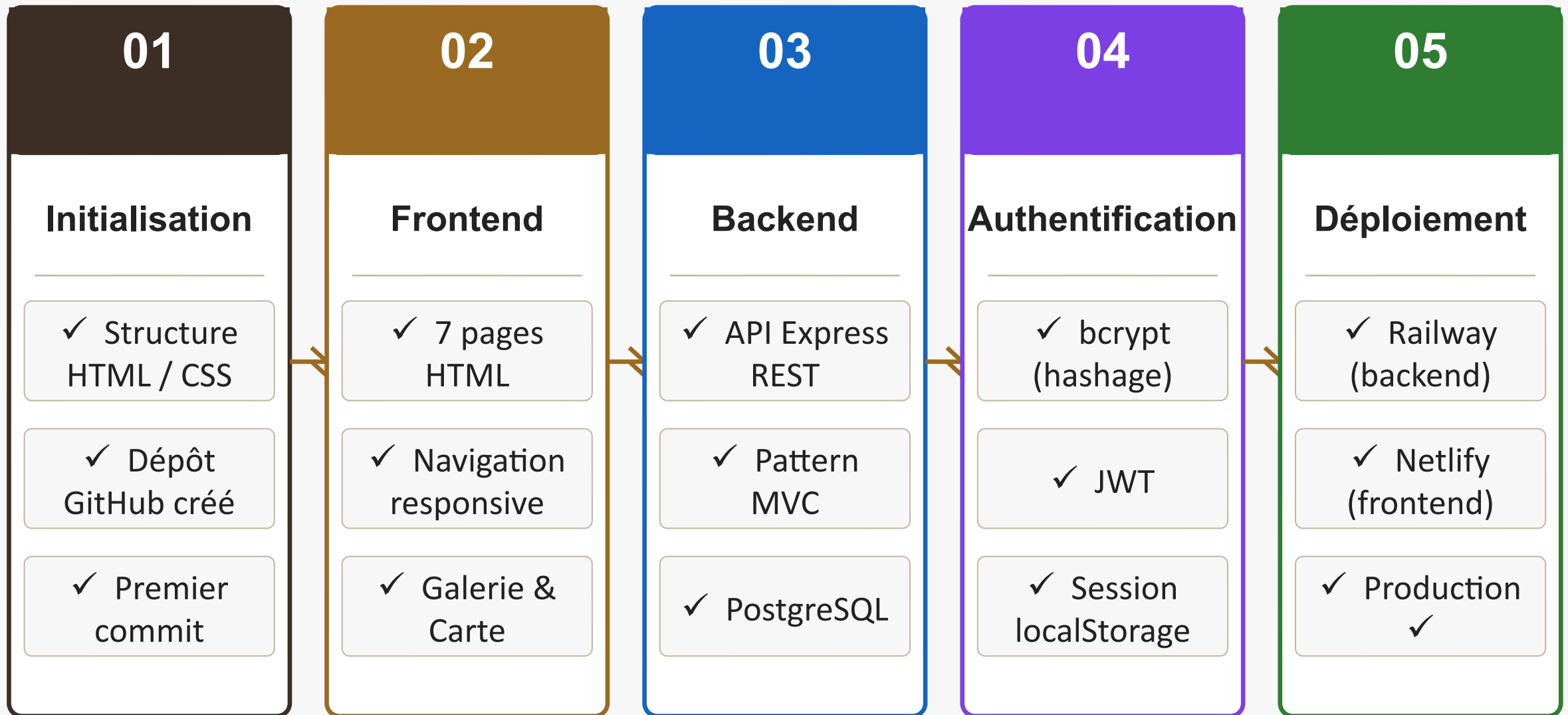
Interface & API

- ✓ Navigation entre les onglets de la carte
- ✓ Affichage sur mobile (390px)
- ✓ Actualisation du dashboard admin
- ✓ Vérification de la route /api/health
- ✓ Test des routes POST via le formulaire de production

11

Bilan & Évolution du projet

Gestion du projet avec Git



✓ 20 commits réalisés · ✓ Historique Git complet · ✓ Développement progressif et documenté

Évolution — Formulaire de réservation

Formulaire de réservation — AVANT → APRÈS

Évolution du projet Quai Antique · Soutenance DWWM

AVANT

Quai Antique

AccueilGalerieLa carteLes reservationsMon compte

Réserver votre repas

Date

dd/mm/yyyy

Heure

--:--

Nombre de personnes

2 personnes

Allergies

Message

Réserver

Nos horaires

Du mardi au dimanche 12:00-14:00 18:00-23:00

Quai Antique

5 rue de la route
31150 Ville
01 61 61 61 61

Contact

contact@quaiautique

v1 · 19 mai — formulaire basique

APRÈS

Quai Antique

AccueilGalerieLa carteLes reservationsMon compte

Réserver votre repas

Prénom

Nom

Email

Date

dd/mm/yyyy

Heure

--:--

Nombre de personnes

2 personnes

Allergies

Message

v3 · 20 mai — connecté à l'API

AVANT : 4 champs · statique · aucune API → APRÈS : 7 champs · fetch() · PostgreSQL · préremplissage

52 / 60

Évolution — Carte du restaurant

Carte du restaurant — AVANT (placeholders) → APRÈS (vrais plats)
Évolution du projet Quai Antique - Soutenance DWWM

AVANT

Quai Antique

AccueilGalerieLa carteLes reservationsMon compte

La carte

Réserver une table

EntréesPlatsDesserts

Entrées

Entrée 18 €

Description du plat

Entrée 210 €

Description du plat

APRÈS

Quai Antique

AccueilGalerieLa carteLes reservationsMon compte

La carte

Réserver une table

EntréesPlatsDesserts

Entrées

Velouté de potimarron aux éclats de noisettes9 €

Soupe onctueuse de potimarron, crème légère, noisettes torréfiées et touche de muscade.

Salade de chèvre chaud au miel11 €

Salade verte, toast de chèvre chaud, miel, noix et vinaigrette maison.

Nos horaires

Du mardi au dimanche 12:00-14:00 18:00-23:00

Quai Antique

5 rue de la route
31150 Ville
04 01 01 01 01

Contact

contact@quaiantique.com

v1 - 19 mai — placeholders

Nos horaires

Du mardi au dimanche 12:00-14:00 18:00-23:00

Quai Antique

5 rue de la route
31150 Ville
04 01 01 01 01

Contact

contact@quaiantique.com

v3 - 27 mai — vrais plats + prix

AVANT : placeholders génériques → APRÈS : vrais plats gastronomiques avec descriptions et prix réels

53 / 60

Fonctionnalités livrées

✓ Vitrine restaurant (accueil, galerie, footer)

✓ Carte avec onglets dynamiques (Entrées / Plats / Desserts)

✓ Formulaire de réservation connecté à PostgreSQL

✓ Inscription sécurisée (hash bcrypt)

✓ Connexion avec authentification JWT

✓ Navigation adaptative (session localStorage)

✓ Préremplissage automatique du formulaire

✓ Dashboard administrateur des réservations

✓ Indicateurs de statut colorés (En attente / Confirmée)

✓ Responsive design mobile et desktop

✓ CORS multi-origine sécurisé

✓ CI/CD automatique GitHub → Netlify + Railway

✓ Docker pour exécution locale

✓ Tests fonctionnels réalisés

✓ Déploiement production opérationnel

✓ Documentation technique complète

Difficultés rencontrées & Solutions apportées

Déploiement Railway — serveur inaccessible en production

✓ Variable PORT · EXPOSE 3000 · switch vers Nixpacks · health endpoint

URL API hardcodée (localhost) — non fonctionnel en ligne

✓ Création de config.js avec l'URL Railway · API_BASE_URL dans tout app.js

Formulaire de réservation vide après connexion

✓ Lecture localStorage au chargement de la page · préremplissage automatique

Compétences développées

Analyse du besoin client

Identification des besoins visiteur, utilisateur et administrateur

Conception de base de données

Modèle ERD · 2 tables · contraintes SQL · initialisation automatique

Développement frontend

7 pages HTML/CSS/JS · responsive · onglets dynamiques · formulaires

Développement backend

API REST Express · MVC · controllers · repositories · routes

Authentification JWT

bcrypt · JWT · localStorage · session côté client

Création d'API REST

5 routes actives · méthodes GET et POST · réponses JSON

Déploiement cloud

Netlify (frontend) · Railway (backend) · Supabase (BDD)

Gestion Git / GitHub

20 commits · CI/CD automatique · historique documenté

Résolution de problèmes

Debug Railway · refactoring URLs · fix UX post-login

12

Compétences DNC validées par le projet

Compétences DNC — Correspondance référentiel

Compétence DNC	Preuve dans le projet	Fichier(s) clé	✓
Concevoir des interfaces	7 pages HTML responsives · charte graphique CSS	styles.css · HTML	✓
Développer des interfaces statiques	Pages accueil, galerie, menu, footer complet	index.html · gallery.html	✓
Développer des interfaces dynamiques	Onglets menu · préremplissage · updateNav()	app.js	✓
Intégrer des formulaires	3 formulaires : réservation, login, register	reservation.html · login.html	✓
Connecter une interface à une API	fetch() vers Railway API (POST/GET)	app.js · config.js	✓
Mettre en place une base de données	PostgreSQL Supabase · 2 tables · SQL préparé	repositories/ · initDatabase.js	✓
Gérer l'authentification	bcrypt (hashage) + JWT (token) + localStorage	auth.controller.js · app.js	✓
Déployer une application web	Netlify + Railway + CI/CD GitHub automatique	railway.json · Dockerfile	✓
Documenter le projet	README · docs/ · diagrammes techniques	docs/ · README.md	✓
Bonnes pratiques sécurité	.gitignore · .env · CORS · requêtes préparées	Backend complet	✓

✓ 10 compétences DNC démontrées et validées par le projet Quai Antique

Conclusion

- ✓ J'ai conçu et développé un site web gastronomique de A à Z
- ✓ J'ai structuré le backend selon le pattern MVC (Routes → Controllers → Repositories)
- ✓ J'ai sécurisé les accès utilisateur avec bcrypt et JWT
- ✓ J'ai connecté un frontend à une API REST et une base de données PostgreSQL
- ✓ J'ai déployé l'application en production sur Netlify et Railway
- ✓ J'ai géré l'intégralité du projet avec Git et un CI/CD automatique



Liens & Ressources

Site Netlify

`https://quai-antique-projet.netlify.app/`

API Railway

`https://quaiantiqueproject-production.up.railway.app`

Dépôt GitHub

`github.com/catcodecat/Quai\_Antique\_Project`

Compte démo

`demo@quaiantique.com · mot de passe : demo123`